

01Display Offline HTML in Android App - Google Gemini

<https://gemini.google.com/share/b168aca37eb7>

Thanks. It worked this time. Thanks for help. Can you give me universal MainActivity.kt file in which I can add any url either offline or online it will still show in the android app without any issue please. It should be able to run all the javascript, html, css etc without any errors. Thanks. Show me the step by step what I need to make it work when I want to use in any new android app creation.

Glad to hear that worked! To prepare your project for any future application—whether it's an offline web application or a live website URL—you can implement a "Universal" solution.

This configuration handles complex desktop-to-mobile responsive layouts, activates JavaScript execution engines, unlocks file permissions, handles modern data caches, and processes mixed online/offline content dynamically.

Part 1: The Universal MainActivity.kt File

This code includes a variable at the top (TARGET_URL). You can set it to a local asset file path or a live link (https://...), and it will execute it correctly.

Kotlin

```
package com.example.jsonextractor02
```

```
import android.annotation.SuppressLint
import android.os.Bundle
import android.webkit.WebChromeClient
import android.webkit.WebSettings
import android.webkit.WebView
import android.webkit.WebViewClient
import androidx.activity.ComponentActivity
import androidx.activity.compose.setContent
import androidx.activity.enableEdgeToEdge
import androidx.compose.foundation.layout.fillMaxSize
import androidx.compose.foundation.layout.padding
import androidx.compose.material3.Scaffold
import androidx.compose.runtime.Composable
import androidx.compose.ui.Modifier
import androidx.compose.ui.viewinterop.AndroidView
import com.example.jsonextractor02.ui.theme.JsonExtractor02Theme
```

```
class MainActivity : ComponentActivity() {
```

```
    // UNIVERSAL CONTROL SWITCH
```

```
    // Change this to whatever website or local asset you want to load!
```

```
    private val TARGET_URL = "file:///android_asset/json-path-extractor.html"
```

```

// Example for a live website link:
// private val TARGET_URL = "https://www.google.com"

override fun onCreate(savedInstanceState: Bundle?) {
    super.onCreate(savedInstanceState)
    enableEdgeToEdge()
    setContent {
        JsonExtractor02Theme {
            Scaffold(modifier = Modifier.fillMaxSize()) { innerPadding ->
                UniversalWebView(
                    url = TARGET_URL,
                    modifier = Modifier.padding(innerPadding)
                )
            }
        }
    }
}

@SuppressLint("SetJavaScriptEnabled")
@Composable
fun UniversalWebView(url: String, modifier: Modifier = Modifier) {
    AndroidView(
        modifier = modifier.fillMaxSize(),
        factory = { context ->
            WebView(context).apply {
                // 1. Keeps link redirects and window loads strictly inside your app screen
                webViewClient = WebViewClient()
                webChromeClient = WebChromeClient()

                // 2. Configure full engine browser properties
                settings.apply {
                    javaScriptEnabled = true    // Run complex JS scripts
                    domStorageEnabled = true    // Allow cookies / theme memory
                    databaseEnabled = true      // Run modern HTML5 DB storage operations

                    // Layout scaling engines (forces 100vh elements to resize accurately)
                    useWideViewPort = true
                    loadWithOverviewMode = true

                    // Unlocks native local app file directory environments
                    allowFileAccess = true
                    allowContentAccess = true

                    @Suppress("DEPRECATION")
                    allowUniversalAccessFromFileURLs = true
                }
            }
        }
    )
}

```

```

        @Suppress("DEPRECATION")
        allowFileAccessFromFileURLs = true

        // Allows rendering external fonts/scripts while browsing local HTML documents
        mixedContentMode = WebSettings.MIXED_CONTENT_ALWAYS_ALLOW
    }

    // 3. Force Android system layout frames to obey physical display matches
    layoutParams = android.view.ViewGroup.LayoutParams(
        android.view.ViewGroup.LayoutParams.MATCH_PARENT,
        android.view.ViewGroup.LayoutParams.MATCH_PARENT
    )

    // 4. Pass the configured URL to the screen view engine
    loadUrl(url)
}
}
)
}

```

Part 2: Blueprint Guide for Building New Web Apps

When building a new app from scratch in Android Studio, follow these steps to ensure everything functions properly:

Step 1: Android Studio Project Generation Setup

Launch Android Studio and click New Project.

Select Empty Activity (the option featuring the small square rainbow Compose symbol). Click Next.

Name your app, select Kotlin as your code programming standard, and complete initialization.

Step 2: Declare Permissions inside the Manifest File

Before testing online web resources or allowing web script execution packages, you must enable network privileges:

Open app > manifests > AndroidManifest.xml.

Place this permission configuration directly above your <application> bracket block:

XML

```
<uses-permission android:name="android.permission.INTERNET" />
```

Step 3: Setup Local Assets Folder (For Offline Pages Only)

If your web resources run locally rather than through a public domain URL:

Look at your folder panel, right-click on the main structural root app directory folder.

Choose New > Folder > Assets Folder and tap Finish.

Drag your HTML file along with any local CSS/JS dependency subfolders straight into that newly generated assets folder.

Step 4: Connect the Code Architecture

Open app > java > your.package.name > MainActivity.kt.

Erase the pre-generated template text completely and paste the universal layout script block provided above.

Update the TARGET_URL variable to point to your link or local filename:

Local file format: "file:///android_asset/yourfile.html"

Live web link format: "https://example.com"

Step 5: Clean Code Theme Adjustments

Look at lines 22 & 24 in the copy-paste file. You will see structural styling theme wrappers named JsonExtractor02Theme.

When you generate a brand new project, your original custom layout name updates matching your project design title (such as MySuperCoolAppTheme).

Simply change JsonExtractor02Theme to whatever design configuration your new project imports inside the document defaults!

Hit Run on your device to launch your web view wrapper app.